

Exercice 1 *Rappels sur la classe Object*

1.1 Précisez les affichages (1) à (9) générés par le code suivant. Expliquez.

```
public class ExecutableObject
{
    public static void main(String[] args)
    {
        Integer x = 3;
        Integer y = new Integer(3);
        System.out.println(x == y); // (1)
        System.out.println(x.equals(y)); // (2)
        System.out.println(x.toString()); // (3)
        System.out.println(x); // (4)
        System.out.println(x.hashCode() == y.hashCode()); // (5)

        List<Object> tableau = new ArrayList<>();
        List<Object> tab2 = new ArrayList<>();
        tableau.add(5);
        tableau.add(5.);
        tableau.add(true);
        tableau.add(tab2);
        System.out.println(tab2); // (6)
        System.out.println(tableau); // (7)

        tableau = new ArrayList<>();
        tab2 = new ArrayList<>();
        tableau.add(4);
        tab2.add(new Integer(4));
        System.out.println(tab2 == tableau); // (8)
        System.out.println(tab2.equals(tableau)); // (9)
    }
}
```

1.2 On vous donne la classe suivante, précisez et expliquez les affichages de (1) à (4)

```
public class Couple
{
    private int premier;
    private int second;
    public Couple(int premier, int second) {
        this.premier = premier;
        this.second = second;
    }
    @Override
    public String toString() {
        return "(" + this.premier + ", " + this.second + ") ";
    }
}
```

Puis l'exécutable

```
public class ExecutableCouple
{
    public static void main(String[] args)
    {
        Couple couple1 = new Couple(4, 5);
        Couple couple2 = new Couple(2 * 2, 3 + 2);
        System.out.println(couple1 == couple2 );//1
        System.out.println(couple1.equals(couple2));//2
        Object couple = couple2;
        System.out.println(couple);//3
        System.out.println(couple.equals(couple2));//4
    }
}
```

1.3 Pour un jeu de SpaceInvaders on a défini une classe Projectile de la manière suivante :

```
public class Projectile
{
    private int positionX, positionY;
    public Projectile(int posX, int posY)
    {
        this.positionX = posX;
        this.positionY = posY;
    }

    @Override
    public String toString()
    {
        return "Projectile à la position :  

        (" + this.positionX + "," + this.positionY + ") ";
    }
}
```

Précisez les affichages (1) à (4) générés par le code suivant. Expliquez.

```
public class ExecutableProjectile
{
    public static void main(String[] args)
    {
        Projectile projectile1 = null;
        Projectile projectile2 = new Projectile(10,10);
        Projectile projectile3 = new Projectile(10,10);
        System.out.println(projectile1);//1
        System.out.println(projectile2);//2
        System.out.println(projectile2 == projectile3);//3
        System.out.println(projectile2.equals(projectile3));//4
    }
}
```

1.4 Comment faire pour que la méthode equals de Projectile fonctionne comme nous le voudrions ?

Exercice 2 *Fantômes et Maison hantée*

- 2.1 Définir une classe `Fantome` dont les attributs sont le nom, la force et la taille (en int).
- 2.2 Définir une classe `MaisonHantee` dont les attributs sont : le nom de celle-ci et l'ensemble des fantômes qui y habitent
- 2.3 Déterminer quel est le fantôme (son nom) le plus fort.
- 2.4 Déterminer l'ensemble des fantômes ayant une force donnée.
- 2.5 Écrire une classe `ExecMaisonHantee` permettant de tester ces méthodes, en voici un extrait

```
public class ExecMaisonHantee {  
    public static void main(String [] args) {  
        Fantome revenant = new Fantome("Le Revenant", 3, 120);  
        Fantome spectre = new Fantome("Le Revenant", 3, 120);  
        Fantome macheur = new Fantome("Le Mâcheur", 1, 190);  
        Fantome frappeur = new Fantome("Le Frappeur", 5, 250);  
        Fantome vortex = new Fantome("Le Vortex", 4, 300);  
        Fantome mortitia = new Fantome("Mortitia", 5, 230);  
        MaisonHantee halloween = new MaisonHantee("Halloween");  
        halloween.ajouter(revenant);  
        halloween.ajouter(spectre);  
        halloween.ajouter(macheur);  
        halloween.ajouter(frappeur);  
        halloween.ajouter(vortex);  
        halloween.ajouter(mortitia);  
        System.out.println(halloween);  
        // la suite  
    }  
}
```

- 2.6 Que constatez-vous sur le dernier affichage ? Pourquoi ? Que proposez-vous ?
- 2.7 Écrire une méthode permettant de trier les fantômes de la maison hantée par ordre de taille décroissante (ordre naturel) et puis de les trier par force croissante (ordre induit). Tentez de n'écrire qu'une seule méthode de tri avec en paramètre le type d'ordre et utilisez une énumération pour chacun des deux ordres (NATUREL, INDUIT).

Exercice 3 *Pokedex*

On vous propose la modélisation d'un pokedex sous la forme d'un dictionnaire dont les clefs sont les noms des attaques (via une énumération `DRAGON`, `EAU`, `SOL`) et les valeurs l'ensemble des noms de pokemon possédant cette attaque

```
pokedexAnakin = {DRAGON = ["Carmache", "Colimucus", "Palkia"],  
                 SOL = ["Carmache"], EAU = ["Palkia"]};
```

- 3.1 Écrire une méthode `appartient` retournant vrai si le nom d'un pokemon appartient au pokedex, faux sinon.
- 3.2 Écrire une méthode `nombreDe` retournant le nombre de pokemons qui possède un type d'attaque donné.
- 3.3 Écrire une méthode `toutesLesAttaques` retournant l'ensemble des types d'attaque d'un pokemon donné. Une exception `PasUnTelPokemonException` sera levée si le pokemon n'appartient pas au pokedex.
- 3.4 Écrire une méthode `attaquePreferee` retournant l'attaque préférée d'un pokedex.
- 3.5 Transformer ce pokedex en un dictionnaire dont les clefs sont les noms des pokemons et les valeurs l'ensemble de leurs attaques.